0 1 2 3 4 5
[1, 5, 7, 2, 3, 4]

[3, 4, 1] ✓ [4, 1] ✓

[1, 4] ✗

arr [i]

arr [i % n]

i	i % n
i=0	0
1	1
2	2
3	3
4	4
5	5
→ 6	0

n = 6

[8, -8, 9, -9, 10, -11, 12] o/p: 22

BF TC = O(n²) ✓

✗ [8, -8, 9, -9, 10, -11, 12, 8, -8, 9, -9, 10, -11, 12]

[1, 2, 3] → [1, 2, 3, 1, 2, 3]

[8, -8, 9, -9, 10, -11, 12].

lowest subarray sum = -11

[8, -8, 9, -9, 10, -11, 12]

minTillHere: 8 -8 1 -9 1 -11 1

global Min = ~~∞~~ ~~8~~ ~~-8~~ ~~-9~~ -11

case-1: Max circular subarray sum lies in a linear subarray.

→ Use normal Kadane's algo & find the max linear subarray sum = S1

case-2: Max circular subarray sum has some elements from the last & some from the beginning.

→ Find the Min. Subarray sum using Kadane's algo = S2

return max (S1, ArraySum - S2)

(

[-1, -1, -1, -1, -1].

TC: O(n)

AS: O(1)

	0	1	2	3	4	5	6
	1,	-2,	3,	4,	-2,	5,	-3
→ start:	0	0	2	2	2	2	2
maxTillHere:	1	-1	3	7	5	$\boxed{10}$	7

global Max: ~~-∞~~ ~~1~~ ~~3~~ ~~7~~ 10

- start =
- end =

0 1 2 3 4 5 6
[1, -2, 3, 4, -2, 5, -3]

mTh = ~~1~~ -1 ~~3~~ ~~7~~ ~~5~~ ~~10~~ 7

currStart = ~~0~~ ~~0~~ ~~2~~ ~~2~~ ~~2~~ 2

~~[start = -1 o/p, end = -1 0]~~

TC = O(n)

AS = O(1)

global Max: ~~-∞~~ ~~1~~ ~~3~~ ~~7~~ 10

st = ~~0~~ 2

en = ~~0~~ ~~2~~ ~~3~~ 5

[a₀, a₁, a₂, a₃ . . . . , aₙ₋₁]

Suffix ~~Prefix~~: aᵢ . . . aₙ₋₁

~~Suffix~~ Prefix: a₀ . . . . aᵢ

Prefix Sum / Suffix Sum
PS SS

$SPS[i] = \sum_{i=i}^{n-1} a_i = a_i + a_{i+1} + a_{i+2} \dots + a_{n-1}$

$PSS[i] = \sum_{i=0}^{i} a_i = a_0 + a_1 + a_2 \dots + a_i$

[1, 3, 5, 2, 4]

PSS = [1, 4, 9, 11, 15] PSS[i] = PSS[i-1] + aᵢ

SPS = [15, 14, 11, 6, 4] SPS[i] = SPS[i+1] + aᵢ

[1, 15] = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 - - - 15

[4, 8] = 4, 5, 6, 7, 8

[3, 5] = 3, 4, 5

[1, 4] = 1, 2, 3, ~~4~~

o/p = 3

freq.

Hash Map - Based.

TC: O(n * maxx)

AS: O(maxx)

→ [3, 5].

0	1	2	3	4	5	6	7		15
			+1			-1		- - - -	
0	0	0	1	1	1	0	0	- - - - - -	0

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
		~~+1~~ 2		+1	~~+1~~ 0			-1		-1							-1
→	0	2	2	3	3	3	2	2	2	1	1	1	1	1	1	1	0

TC = O(n + maxx)

AS = O(maxx).

10⁸ operations in 1 sec.

[1, 2, 3, 4]

[24, 12, 8, 6]

nums: [10⁹, 10⁹]

ans: [10⁹, 10⁹]

[1, 2, 3, 4].

prefProd [i] = a₀ × a₁ × - - - × aᵢ₋₁

suffProd (i) = aᵢ₊₁ × aᵢ₊₂ × - - - - × aₙ₋₁

ans [i] = prefProd [i] × suffProd [i].

	[1,	2,	3,	4].
PP:	1	1	2	6
SP:	24	12	4	1
ans:	[24,	12,	8,	6]

[1, 2, 4, 8], target = 8

o/p = False.

[1, 2, 3, 4, 8] target = 8.

o/p: True.

```
for (int i = 0; i < n-1; i++) {
    Linear Search for (target - arr[i]) in arr[i+1 - - - n-1]
}
```

TC = O(n²)

AS = O(1)

```
for (int i = 0; i < n-1; i++) {
    ~~Linear~~ Binary Search for (target - arr[i]) in arr[i+1 - - - n-1]
}
```

~~TC = O~~) TC = O(n log n)

AS = O(1).

HashMap Based. → TC = O(n), AS = O(n)

(0 - - - - - - i-1) i

[1, 2, 5, 6, 10], tar = 8

Hash Map.

1 → True.

→ 2 → True.

5 → True.